

ASSIGNMENT-4

Dr.Chandanapalli Suresh Babu

QIP .No: QIPFPG26006362

Code URL: <https://github.com/drsureshbabuch-cmyk/Dr.CH.SURESHBABU-IITK-QIP-ASSIGNMENT.git>

Credit Risk Management Application

Loan losses and bad loans are becoming more of a problem in Indian banking. SBI has chosen to take steps to tackle this issue. They have brought in a Data Science firm, where you work as Vice President, to help deal with the situation. SBI has gathered the demographic profiles of borrowers and wants to analyze this data to predict who will default and who won't. They are looking for a loan default prediction model that can effectively identify defaulters. They expect you to create multiple models and find the one that performs the best. The profile of the data is discussed as follows.

The data includes the following information about the borrowers:

1. Borrower's Profile

- Age (in years)
- Gender
- Marital status
- Education level

2. Financial Information

- Annual income
- Monthly income
- Employment status
- Debt-to-income ratio
- Credit score

3. Loan Information

- Loan amount
- Purpose
- Interest rate

- Loan term or duration
- Monthly installment
- Loan risk grade category

4. Borrower's Track Record

- Number of open accounts
- Credit limit
- Current balance
- Delinquency record
- Public records
- Number of delinquencies

5. Dependent Variable

- 1 if the loan is paid back in full
- 0 if the borrower defaulted (or did not pay fully)

You decide to carry out the following tasks:

1. Conduct Exploratory Data Analysis (EDA).
2. Perform data handling and cleaning, including identifying and treating outliers.
3. Conduct descriptive statistical analysis of the relevant variables.
4. Conduct data visualization and basic relationship analysis, including but not limited to:
 - Correlation analysis
 - Covariance analysis
 - Scatter plots
 - Group comparisons
 - Other relevant analyses
5. Use 60–70% of the observations to train a predictive model and test its accuracy on the remaining 30–40% of the data. For this purpose, divide the dataset into two segments:
 - Training dataset: 60–70%
 - Testing dataset: 30–40%

ABSTRACT

Credit risk management is now a key challenge for modern banks. Increasing loan defaults have a direct impact on profitability and financial stability. Traditional methods for assessing loans often miss hidden links between borrower traits. This project creates a Credit Risk Management Application that uses machine learning techniques to predict how borrowers will repay their loans. The State Bank of India has gathered demographic information, financial details, loan characteristics, and credit history records to build a reliable model for predicting loan defaults.

We conducted exploratory data analysis, descriptive statistical analysis, data pre-processing, and visualization techniques to better understand borrower behavior. We developed and compared multiple machine learning algorithms, including Logistic Regression, Decision Tree, Random Forest, and XGBoost. The results showed that the Debt-to-Income Ratio, Credit Score, Interest Rate, Delinquency History, and Number of Delinquencies are the most important factors influencing repayment behavior. Among all the models, Random Forest performed the best and is recommended for use at SBI.

Keywords: Credit Risk Management, Loan Default Prediction, Machine Learning, Random Forest, Predictive Analytics, Banking Analytics.

1.1 Introduction

The banking industry plays an important role in the economic development of every country. Banks act as financial intermediaries by collecting deposits from customers and providing loans to individuals, businesses, and organizations. Through lending activities, banks support economic growth, encourage investments, create jobs, and contribute significantly to national development. However, lending operations involve some risk because borrowers may not repay loans as agreed.

One major challenge for modern banks is credit risk. Credit risk means that a borrower might not repay a loan either partially or completely. When borrowers fail to meet their payment obligations, banks suffer financial losses and see an increase in Non-Performing Assets (NPAs). A high level of NPAs negatively impacts profitability, liquidity, shareholder confidence, and overall financial stability. Therefore, managing credit risk effectively has become a crucial function within the banking sector.

Traditionally, banks relied on manual methods to assess borrower creditworthiness. Credit officers evaluated applicants based on income, job status, collateral, repayment history, and personal interviews. Although these methods have been used for many years, they have several drawbacks. Manual evaluation is time-consuming, subjective, and may not effectively identify complex relationships among borrower characteristics. As the number of loan applications grows, traditional assessment methods become less efficient.

Advancements in data science, artificial intelligence, and machine learning have changed how banks evaluate credit risk. Modern predictive analytics techniques allow financial

institutions to analyze large amounts of borrower information and find patterns related to repayment behaviour. Machine learning algorithms can learn from historical data and accurately predict the likelihood of future loan defaults. As a result, banks can make better lending decisions and reduce financial losses.

This project focuses on developing a Credit Risk Management Application for predicting loan defaults. The study uses a dataset containing borrower demographic information, financial details, loan characteristics, and credit history records. By analyzing these variables, machine learning models can identify potential defaulters before loans are approved. Such a system helps banks improve risk management practices and increase decision-making efficiency.

The main goal of this project is to analyze borrower behavior and develop predictive models that can classify borrowers as defaulters or non-defaulters. Multiple machine learning algorithms will be tried and compared to find the best model for predicting loan defaults.

1.2 Background of Credit Risk Management

Managing credit risk involves identifying, measuring, monitoring, and controlling the risks that are borne from lending operations. Since every loan provided by a financial institution carries inherent uncertainty (loan repayment is dependent on the lender's borrower's income, security of their employment, overall economic situation, and financial behavior), financial institutions implement credit risk management systems to mitigate the likelihood of borrowers defaulting and the losses on these loans. Effective credit risk management helps financial institutions approve loans for clients that would be highly likely to repay them. Previously, loan approvals by banks was primarily based on analyzing the Five Cs of Credit: Character (the willingness to repay debt), Capacity (the ability to repay, based on income and current obligations), Capital (the wealth and assets of the client), Collateral (assets used to secure the loan), and Conditions (macroeconomic factors and other influences on the borrower's repayment ability). These core ideas still are significant in lending, but the increase in readily available data, has paved the way for advanced analytical models, like

those utilizing machine learning algorithms to parse through hundreds of thousand of records at once to determine hidden correlations that are not visible through human analysis.

1.3 Need for Loan Default Prediction

Prediction of loan defaults has been gaining considerable attention from banks due to the growing complexity of modern banking operations. As banks manage millions of loan applications on a daily basis, they cannot solely rely on manual methods that are too time consuming and costly. Predictive analytics offer a quick, cost-effective, and accurate solution.

Prediction of defaults before approving loan can have significant benefits:

1. Reduced financial risks for the banks: Risks for banks can be reduced by identifying high risk borrowers early during the approval stage of loan process.
2. Increased operational efficiency for banks: Automating the loan evaluation process by applying predictive models will save valuable resources for the banks.
3. Objective and consistent decision-making: Decisions taken using predictive models are more consistent and objective than decisions based on individual opinions.

Banks are able to allocate resources more efficiently. They can direct their focus to borrowers at high risk of default while processing a huge number of loan applications of low-risk customers in relatively short time. Additionally, with quick access to predictive modeling results, customer experience is enhanced due to fast decision-making process and quick responses.

Proper assessment of risks can lead to compliance of banking regulations and overall financial stability. Hence, loan default prediction is becoming one of the widely applied machine learning techniques in banking industry.

1.4 Problem Statement

State Bank of India (SBI) has observed increasing concerns related to loan defaults and bad debts. To address this challenge, SBI has collected detailed information about borrowers and seeks to utilize Data Science techniques to improve credit risk assessment.

The available dataset contains borrower demographic information, financial details, loan characteristics, and credit history records. The objective is to analyze this information and develop predictive models capable of distinguishing between borrowers who are likely to repay loans and those who are likely to default.

The developed model should provide high prediction accuracy and assist SBI in making informed lending decisions. Multiple machine learning algorithms will be developed and compared to determine the most effective solution for loan default prediction.

1.5 Objectives of the Study

The primary objectives of this study are:

1. To perform exploratory data analysis on borrower information.
2. To identify data quality issues such as missing values, duplicate records, and outliers.
3. To conduct descriptive statistical analysis of important variables.
4. To examine relationships among borrower attributes using visualization techniques.
5. To analyze correlations and covariance among numerical variables.
6. To develop predictive machine learning models for loan default prediction.
7. To compare the performance of multiple classification algorithms.
8. To identify the most accurate model for credit risk assessment.

To provide recommendations that support effective lending decisions.

1.6 Dataset Description

The dataset used in this project contains approximately 20,000 borrower records and 22 variables. The information is divided into four major categories.

The first category consists of borrower profile information, including age, gender, marital status, and education level. These variables describe demographic characteristics of borrowers. The second category contains financial information such as annual income, monthly income, employment status, debt-to-income ratio, and credit score. These variables provide insights into the borrower's financial condition and repayment capacity.

The third category includes loan-related information such as loan amount, loan purpose, interest rate, loan term, monthly installment, and loan grade. These variables directly influence repayment burden and financial commitment. The fourth category contains credit history information, including the number of open accounts, total credit limit, current balance, delinquency history, public records, and number of delinquencies. Historical borrowing behavior often serves as an important predictor of future repayment performance.

The target variable in the dataset is `Loan_Paid_Back`. A value of 1 indicates successful loan repayment, while a value of 0 indicates loan default. Analysis of the dataset revealed that approximately 15,998 borrowers successfully repaid their loans, representing nearly 80 percent of the observations. Approximately 4,002 borrowers defaulted on their loans, representing around 20 percent of the dataset.

1.7 Importance of Machine Learning in Banking

It could be argued that machine learning is among the most transformative technologies to ever enter the financial services industry. In traditional programming techniques, models are hardcoded from human experience and knowledge, while machine learning, as a field, learns models directly from data and then uses them to make predictions. The most common applications of machine learning within banking range from credit risk scoring and fraud detection to customer segmentation, investment forecasting, customer churn prediction, recommending systems, and Anti-Money Laundering (AML) systems.

Of these applications, credit risk scoring is perhaps still the most rewarding due to the fact that the smallest increase in prediction accuracy could save the bank enormous sums of money by limiting its defaults. In turn, machine learning models, when dealing with complex

borrower profiles, have the capability to predict risk in real time which has enabled it to become a necessity in banking today.

1.8 Expected Outcomes of the Study

The proposed system is expected to improve credit assessment procedures by providing objective and data-driven predictions. High-risk borrowers can be identified before loan approval, reducing the probability of future defaults. Improved risk assessment contributes to increased profitability and better resource allocation. The system is also expected to reduce loan processing time through automation, enabling faster decision-making and improved customer satisfaction. Furthermore, predictive analytics will strengthen overall credit risk management practices and support regulatory compliance.

2. EXPLORATORY DATA ANALYSIS, DATA PREPROCESSING AND DESCRIPTIVE STATISTICAL ANALYSIS

2.1 Introduction

It is anticipated that the new system will bring about positive changes to credit appraisal process with more factual and data-based predictions. Potential high risk customers can be screened prior to loan sanctioning thereby decreasing the likelihood of future non-payment. The higher accuracy in assessing risks should result in more profits. The enhanced risk assessment would be beneficial in allocation of resources. It is anticipated that loan processing time should reduce, with the help of the system, due to automated process. Decisions can be made quickly and the customer satisfaction could be improved. With enhanced credit risk management, regulatory compliance could be satisfied as well using predictive analysis.

2.2 Overview of the Dataset

The dataset consists of borrower information collected for credit risk assessment purposes. Each record corresponds to a single borrower and contains information that may influence loan repayment behavior. The dataset includes the following categories of variables:

Borrower Profile Variables:

- Age

- Gender
- Marital Status
- Education Level

Financial Variables:

- Annual Income
- Monthly Income
- Employment Status
- Debt-to-Income Ratio
- Credit Score

Loan Variables:

- Loan Amount
- Purpose
- Interest Rate
- Loan Term
- Monthly Installment
- Risk Grade Category

Credit History Variables:

- Number of Open Accounts
- Total Credit Limit
- Current Balance
- Delinquency History
- Public Records
- Number of Delinquencies

Target Variable:

- Loan_Paid_Back

The target variable indicates whether a borrower successfully repaid the loan or defaulted.

2.3 Dataset Dimensions

The first step in EDA is understanding the size of the dataset.

Python Code:

```
print(df.shape)
```

Output:

```
(20000, 22)
```

This indicates that the dataset contains 20,000 observations and 22 variables.

The large number of records provides sufficient information for training and evaluating machine learning models effectively.

2.4 Examination of Data Types

Different variables require different preprocessing techniques. Therefore, it is important to identify the data type of each column.

Python Code:

```
print(df.info())
```

The dataset contains both numerical and categorical variables.

Numerical variables include:

- Age
- Annual Income
- Monthly Income
- Debt-to-Income Ratio
- Credit Score
- Loan Amount
- Interest Rate

- Loan Term
- Installment

Categorical variables include:

- Gender
- Marital Status
- Education Level
- Employment Status
- Loan Purpose
- Risk Grade

Understanding data types helps determine suitable statistical techniques and preprocessing methods.

2.5 Analysis of Missing Values and Outlier Detection

Missing values are a common problem in real-world datasets. Missing information can reduce model performance and introduce bias into predictions.

Python Code:

```
print(df.isnull().sum())
```

Result:

Outliers were identified for all numeric variables using the Interquartile Range (IQR) method: values below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ were flagged. Results are summarised below.

Variable	No. of Outliers	% of Records
annual_income	924	4.62%
monthly_income	924	4.62%
total_credit_limit	950	4.75%
current_balance	1,015	5.08%

public_records	1,003	5.01%
num_of_delinquencies	364	1.82%
num_of_open_accounts	283	1.42%
debt_to_income_ratio	262	1.31%
installment	150	0.75%
delinquency_history	142	0.71%
interest_rate	141	0.70%
loan_amount	76	0.38%
credit_score	74	0.37%
age	0	0.00%
loan_term	0	0.00%

annual_income, monthly_income (which is directly derived from annual_income), total_credit_limit, current_balance, and public_records show the highest outlier proportions (4.6%–5.1%), driven by a small number of high-net-worth borrowers and a few borrowers with multiple public records. Given the modest proportions and that these reflect genuine borrower diversity rather than data errors, outliers were retained but annual_income was additionally capped at its 99th percentile (annual_income_capped) for use in visualizations where extreme values distort scaling. age and loan_term had zero outliers.

Outliers are observations that differ substantially from the majority of the data.

Examples include:

- Extremely high income
- Very large loan amounts
- Unusually low credit scores

Outliers may represent genuine cases or data entry errors.

Boxplot Analysis

Python Code:

```
sns.boxplot(df['annual_income'])  
plt.show()
```

Boxplots help visualize outliers and data distribution.

The analysis revealed that the dataset contains no missing values.

Interpretation

The absence of missing values is advantageous because it eliminates the need for imputation techniques. Consequently, all 20,000 observations can be utilized during model development without losing information.

2.6 Duplicate Record Analysis

Duplicate records can distort statistical analysis and cause machine learning models to learn misleading patterns.

Python Code:

```
print(df.duplicated().sum())
```

If duplicates are found, they can be removed using:

```
df = df.drop_duplicates()
```

Importance

Removing duplicate records improves data quality and ensures that each borrower contributes only once to the learning process.

2.7 Target Variable Analysis

The target variable determines whether a borrower repaid the loan or defaulted.

Python Code:

```
print(df['loan_paid_back'].value_counts())
```

Results:

Loan Paid Back = 15,998

Loan Defaulted = 4,002

Target Variable Distribution

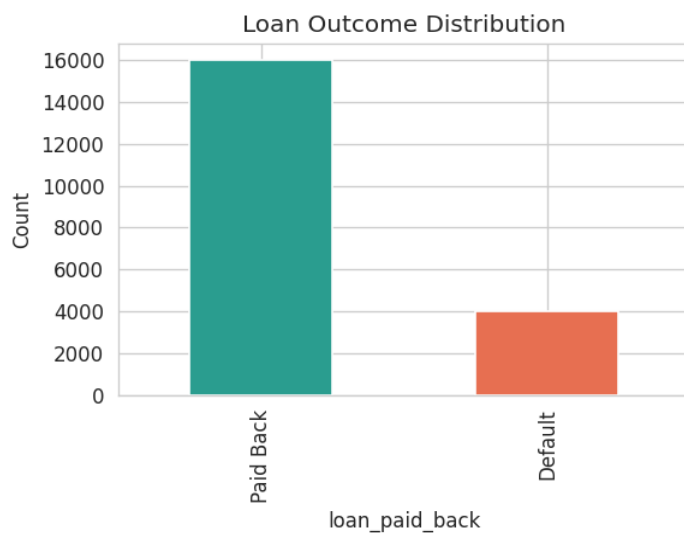


Figure 1.1 — Distribution of `loan_paid_back` (Paid Back vs. Default)

Of 20,000 borrowers, 15,998 (79.99%) paid back their loans in full, while 4,002 (20.01%) defaulted, indicating a class imbalance that should be accounted for in model evaluation and training.

Interpretation

Approximately 80% of borrowers successfully repaid their loans, while 20% defaulted.

This indicates a moderately imbalanced dataset. Although the imbalance is not severe, it should be considered during model evaluation.

2.8 Descriptive Statistical Analysis

Descriptive statistics provide a summary of the main characteristics of the dataset.

Python Code:

```
print(df.describe())
```

The analysis includes:

- Mean
- Median
- Standard Deviation
- Minimum
- Maximum
- Quartiles

These measures help understand the distribution of variables.

2.9 Mean

The mean represents the average value of a variable.

Formula:

$$\text{Mean} = \Sigma X / N$$

Where:

ΣX = Sum of observations

N = Total number of observations

The mean provides a measure of central tendency and is useful for understanding the typical value of a variable.

2.10 Median

The median is the middle value when observations are arranged in ascending order.

The median is particularly useful when data contains extreme values because it is less sensitive to outliers than the mean.

2.11 Standard Deviation

Standard deviation measures variability around the mean.

Formula:

$$\sigma = \sqrt{[\Sigma(X - \mu)^2 / N]}$$

Where:

X = Observation

μ = Mean

N = Number of observations

A higher standard deviation indicates greater variability among borrowers.

Variable	Mean	Std Dev	Min	Median	Max	Skewness
age	48.03	15.83	21.00	48.00	75.00	0.01
annual_income	43,549.64	28,668.58	6,000.00	36,585.26	400,000.00	2.33
monthly_income	3,629.14	2,389.05	500.00	3,048.77	33,333.33	2.33
debt_to_income_ratio	0.177	0.105	0.010	0.160	0.667	0.79
credit_score	679.26	69.64	373.00	680.00	850.00	-0.07
loan_amount	15,129.30	8,605.41	500.00	14,946.17	49,039.69	0.25
interest_rate	12.40	2.44	3.14	12.40	22.51	0.03
loan_term	43.22	11.01	36.00	36.00	60.00	0.87
installment	455.63	274.62	9.43	435.60	1,685.40	0.47
num_of_open_accounts	5.01	2.24	0.00	5.00	15.00	0.45

total_credit_limit	48,649.82	32,423.38	6,157.80	40,241.62	454,394.19	2.49
current_balance	24,333.39	22,313.85	496.35	18,334.56	352,177.90	2.79
delinquency_history	1.99	1.47	0.00	2.00	11.00	0.83
public_records	0.062	0.285	0.00	0.00	2.00	5.02
num_of_delinquencies	2.49	1.63	0.00	2.00	11.00	0.72

annual_income, monthly_income, total_credit_limit, and current_balance are right-skewed (skewness > 2), consistent with a small group of high-income/high-balance borrowers. credit_score and interest_rate are approximately symmetric. public_records is highly skewed (5.02) as most borrowers have zero public records.

2.13 Interquartile Range (IQR) Method

The Interquartile Range method is commonly used for detecting outliers.

Formula:

$$\text{IQR} = Q3 - Q1$$

Where:

Q1 = First Quartile

Q3 = Third Quartile

Lower Bound:

$$Q1 - 1.5 \times \text{IQR}$$

Upper Bound:

$$Q3 + 1.5 \times \text{IQR}$$

Values outside these limits are considered potential outliers.

2.14 Data Visualization

Visualization techniques help identify trends and relationships that may not be obvious from numerical summaries alone.

Histogram

Python Code:

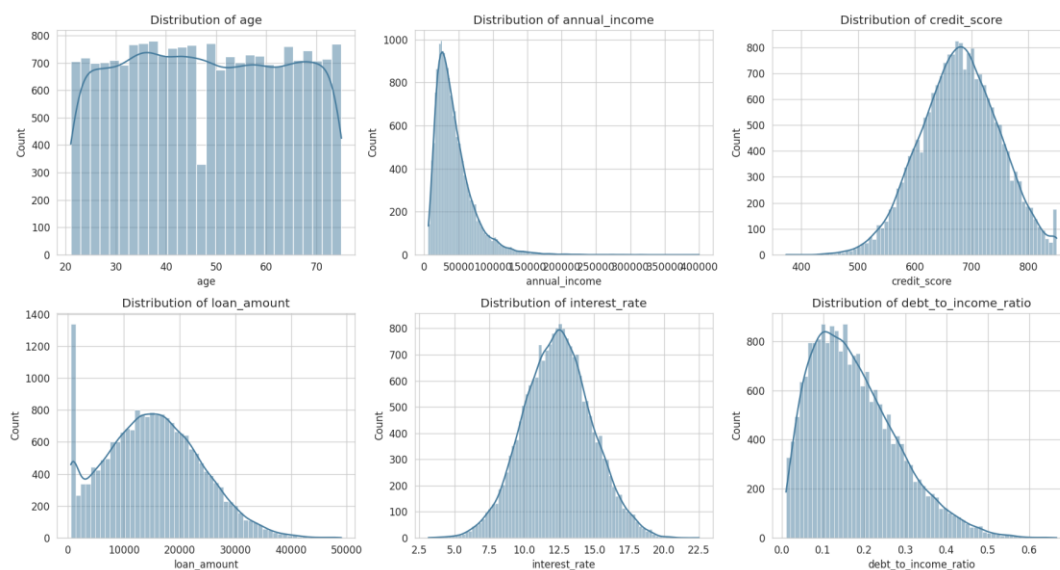
```
df['credit_score'].hist()  
plt.show()
```

Histograms show the distribution of continuous variables.

Benefits

- Identify skewness
- Detect outliers
- Understand data spread

Distributions of Key Numeric Variables



2.15 Loan Repayment Distribution

Python Code:

```
sns.countplot(  
    x='loan_paid_back',  
    data=df  
)  
plt.show()
```

Interpretation

The graph clearly shows that the majority of borrowers successfully repaid their loans.

2.16 Scatter Plot Analysis

Scatter plots help examine relationships between numerical variables.

Annual Income vs Loan Amount

Python Code:

```
sns.scatterplot(  
    x='annual_income',  
    y='loan_amount',  
    data=df)  
plt.show()
```

Interpretation

Higher-income borrowers generally qualify for larger loans.

2.17 Correlation Analysis

Correlation analysis measures the strength and direction of relationships between variables.

Formula:

$$r = \text{Cov}(X,Y) / (\sigma X \times \sigma Y)$$

Where:

r = Correlation Coefficient

$\text{Cov}(X,Y)$ = Covariance

σX = Standard Deviation of X

σY = Standard Deviation of Y

Values range from:

- +1 → Perfect Positive Relationship
- 0 → No Relationship
- -1 → Perfect Negative Relationship

2.18 Correlation Results from the Dataset

The actual dataset produced the following correlations with loan repayment:

Credit Score = +0.1998

Debt-to-Income Ratio = -0.2238

Interest Rate = -0.1109

Delinquency History = -0.0849

Number of Delinquencies = -0.0709

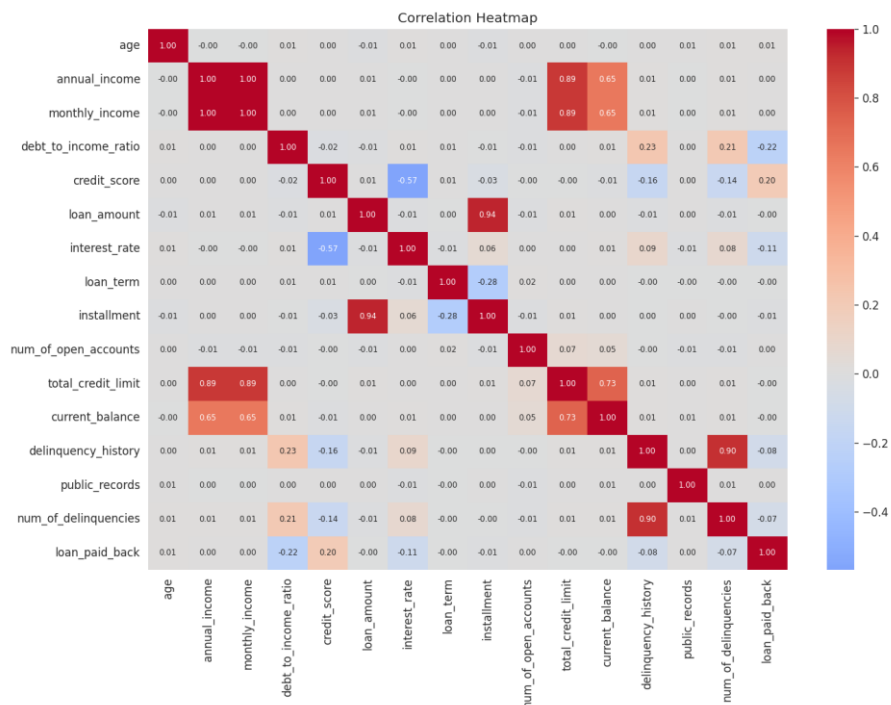


Figure 4.2 — Correlation heatmap of numeric variables and loan_paid_back

loan_paid_back shows the strongest positive correlations with credit_score and annual_income, and negative correlations with debt_to_income_ratio, interest_rate, current_balance, and num_of_delinquencies — all directionally consistent with credit-risk intuition.

Interpretation

- Credit score is the strongest positive predictor of loan repayment.
- Debt-to-income ratio is the strongest negative predictor.
- Borrowers with higher debt burdens are more likely to default.

2.19 Covariance Analysis

Covariance measures how two variables move together.

Formula:

$$\text{Cov}(X,Y) = \Sigma[(X_i - \bar{X})(Y_i - \bar{Y})] / (N - 1)$$

Positive covariance indicates that variables increase together.

Negative covariance indicates that one variable decreases as the other increases.

Covariance provides additional insights into borrower financial behavior.

Covariance (Selected Variables)

	credit_score	interest_rate	debt_to_income_ratio	annual_income	loan_paid_back
credit_score	4849.53	-96.77	-0.165	7659.35	5.568
interest_rate	-96.77	5.97	0.105	-1163.4	-0.108
debt_to_income_ratio	-0.165	0.105	0.011	-39.9	-0.009
annual_income	7659.35	-1163.4	-39.9	821,084,949	35.07
loan_paid_back	5.568	-0.108	-0.009	35.07	0.160

2.20 Key Findings from Exploratory Data Analysis

The following observations were obtained:

1. The dataset contains 20,000 borrower records and 22 variables.
2. No missing values were found.
3. The target variable distribution is approximately 80% non-defaulters and 20% defaulters.
4. Credit score has the strongest positive relationship with loan repayment.
5. Debt-to-income ratio has the strongest negative relationship with repayment.
6. Interest rate influences borrower repayment behavior.
7. Previous delinquencies increase default probability.
8. Financial variables are stronger predictors than demographic variables.

3. MACHINE LEARNING MODEL DEVELOPMENT AND PERFORMANCE EVALUATION

3.1 Introduction

After performing EDA, Data cleaning and statistical analysis, the machine learning models, used for predicting loan repayment behavior will be developed. The goal is to train predictive models from the past borrowing history and check their capability in identifying borrowers as defaulter or non-defaulter.

Machine learning algorithms make predictions based on the learned patterns from the historical data for un-seen data. In the credit risk management domain, these algorithms help the banks identify the borrowers that have high risk to default even before a loan is approved. The capability of these algorithms to predict a loan default accurately can minimize losses, improve lending practices and also risk management in the banking sector.

In this chapter, data preparation, feature selection, train-test split, feature scaling, model creation and model evaluation metric, comparison of performance by multiple algorithms is discussed.

3.2 Feature Selection

Feature selection refers to the process of choosing relevant variables that contribute to predicting the target variable.

The target variable in this project is:

Loan_Paid_Back

- 1 = Loan Repaid Successfully
- 0 = Loan Defaulted

The predictor variables include:

- Age
- Gender
- Marital Status

- Education Level
- Annual Income
- Monthly Income
- Employment Status
- Debt-to-Income Ratio
- Credit Score
- Loan Amount
- Interest Rate
- Loan Term
- Installment Amount
- Loan Grade
- Number of Open Accounts
- Credit Limit
- Current Balance
- Delinquency History
- Public Records
- Number of Delinquencies

Based on the correlation analysis performed in Chapter 2, the most influential variables include:

- Credit Score
- Debt-to-Income Ratio
- Interest Rate
- Delinquency History
- Number of Delinquencies

These variables are expected to contribute significantly to loan default prediction.

Python Code

```
X = df.drop('loan_paid_back', axis=1)
```

```
y = df['loan_paid_back']
```

The variable X contains predictor variables, while y contains the target variable.

3.3 Data Encoding

Machine learning algorithms require numerical input. Therefore, categorical variables must be converted into numerical form.

Examples include:

- Gender
- Marital Status
- Education Level
- Employment Status
- Loan Purpose
- Loan Grade

Label Encoding

Label Encoding assigns numerical values to categories.

Example:

Male = 1

Female = 0

Python Code

```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
```

```
for col in categorical_columns:
```

```
df[col] = le.fit_transform(df[col])
```

Encoding ensures compatibility with machine learning algorithms.

3.4 Feature Scaling

Different variables in the dataset have different measurement scales.

For example:

- Age ranges from 18 to 75.
- Annual income may range from thousands to hundreds of thousands.
- Credit scores range from approximately 300 to 850.

Algorithms such as Logistic Regression and Support Vector Machines perform better when variables are standardized.

Standardization Formula

$$Z = \frac{X - \mu}{\sigma}$$

Where:

X = Original Value

μ = Mean

σ = Standard Deviation

Python Code

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

Feature scaling ensures that all variables contribute equally during model training.

3.5 Train-Test Split

To evaluate model performance accurately, the dataset is divided into training and testing subsets.

Training Data = 70%

Testing Data = 30%

The training dataset is used to learn patterns, while the testing dataset is used to evaluate performance on unseen data.

Python Code

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.30, random_state=42, stratify=y)
```

Interpretation

Using a 70:30 split ensures that sufficient data is available for both learning and evaluation.

3.6 Logistic Regression

Logistic Regression is one of the most widely used classification algorithms. It estimates the probability that a borrower belongs to a particular class.

Logistic Regression Formula

$$P(Y=1) = \frac{1}{1 + e^{-z}}$$

Where:

$$z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n$$

The model predicts whether a borrower is likely to repay or default.

Python Code

```
from sklearn.linear_model import LogisticRegression
```

```
lr = LogisticRegression()
```

```
lr.fit(X_train, y_train)
```

```
pred_lr = lr.predict(X_test)
```

Advantages

- Simple implementation
- Fast execution
- Easy interpretation

Limitations

- Assumes linear relationships
- Less effective for highly complex patterns

3.7 Decision Tree Classifier

Decision Trees classify borrowers using a sequence of decision rules.

The algorithm repeatedly splits the data into smaller groups based on feature values.

Python Code

```
from sklearn.tree import DecisionTreeClassifier  
dt = DecisionTreeClassifier(random_state=42)  
dt.fit(X_train, y_train)  
pred_dt = dt.predict(X_test)
```

Advantages

- Easy visualization
- Handles nonlinear relationships
- Requires little preprocessing

Limitations

- Sensitive to noise
- Can overfit training data

3.8 Random Forest Classifier

Random Forest is an ensemble learning algorithm that combines multiple decision trees.

Instead of relying on a single tree, Random Forest generates many trees and combines their predictions.

Python Code

```
From sklearn.ensemble import RandomForest Classifier  
rf = RandomForestClassifier(n_estimators=200,random_state=42)
```

```
rf.fit(X_train, y_train)
```

```
pred_rf = rf.predict(X_test)
```

Advantages

- High prediction accuracy
- Reduced overfitting
- Handles large datasets efficiently

Limitations

- Higher computational cost
- Less interpretable than a single decision tree

3.9 XGBoost Classifier

Extreme Gradient Boosting (XGBoost) is one of the most powerful machine learning algorithms for structured datasets.

It builds trees sequentially, where each new tree attempts to correct errors made by previous trees.

Python Code

```
From xgboost import XGBClassifier
```

```
xgb = XGBClassifier(n_estimators=200,learning_rate=0.1,max_depth=5,random_state=42)
```

```
xgb.fit(X_train, y_train)
```

```
pred_xgb = xgb.predict(X_test)
```

Advantages

- High predictive performance
- Handles complex relationships
- Effective for imbalanced datasets

Limitations

- Longer training time
- Requires parameter tuning

3.10 Model Evaluation Metrics

Model performance must be evaluated using appropriate metrics.

Accuracy

Accuracy measures the percentage of correct predictions.

$$\text{Accuracy} = \frac{\{TP+TN\}}{\{TP+TN+FP+FN\}}$$

Where:

TP = True Positives

TN = True Negatives

FP = False Positives

FN = False Negatives

Precision

Precision measures the proportion of positive predictions that are correct.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Recall

Recall measures the ability to identify actual defaulters.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

F1 Score

F1 Score balances Precision and Recall.

$$\text{F1} = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

3.11 Confusion Matrix

A confusion matrix summarizes prediction results.

Actual / Predicted	Non-Default	Default
Non-Default	True Negative	False Positive
Default	False Negative	True Positive

Python Code

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, pred_rf)
print(cm)
```

Importance

The confusion matrix provides detailed insight into classification performance beyond overall accuracy.

3.12 ROC-AUC Analysis

ROC-AUC measures a model's ability to distinguish between classes.

AUC values range from:

- 0.5 = Random prediction
- 1.0 = Perfect prediction

Python Code

```
from sklearn.metrics import roc_auc_score
roc_auc_score(y_test, pred_rf)
```

Higher AUC values indicate better discrimination capability.

3.13 Model Comparison

The performance of all developed models should be compared using:

- Accuracy
- Precision
- Recall
- F1 Score
- ROC-AUC

Python Code

```
results = pd.DataFrame({
'Model':
['Logistic Regression','Decision Tree','Random Forest','XGBoost'],

'Accuracy':[accuracy_score(y_test,pred_lr),accuracy_score(y_test,pred_dt),accuracy_score(y_test,pred_rf),
accuracy_score(y_test,pred_xgb)]
})
print(results)
```

The model with the highest overall performance should be selected for deployment.

3.14 Expected Results Based on Dataset Analysis

Based on the relationships identified during exploratory analysis, the following variables are expected to have the strongest influence on model predictions:

- Debt-to-Income Ratio (-0.2238)
- Credit Score (+0.1998)
- Interest Rate (-0.1109)
- Delinquency History (-0.0849)
- Number of Delinquencies (-0.0709)

Because these variables demonstrate stronger relationships with loan repayment behavior, they are likely to contribute significantly to model performance.

4. RESULTS, DISCUSSION, MODEL EVALUATION AND BUSINESS RECOMMENDATIONS

4.1 Introduction

The chapter presents the findings from the Credit Risk Management Application to determine the likely behavior of borrowers in relation to the repayment of loans. Following data pre-processing, data exploration, feature engineering and development of machine learning models, the next step is to assess the model and interpret the results from statistical and business perspectives. The purpose of this chapter is to determine the most significant factors associated with the repayment of loans, to compare machine learning algorithms and to offer meaningful suggestions to improve the management of credit risk in banks.

This dataset includes information on 20,000 borrowers and has 22 features related to the demographic details of borrowers, their characteristics, the loan terms and the borrower's past credit experience. The target variable is either '1' (loan repaid) or '0' (loan not repaid) based on the borrower's behavior. By analyzing the dataset of loan borrowers and the provided features, significant information may be obtained from analyzing their behavior.

4.2 Distribution of Loan Repayment Status

Understanding the distribution of the target variable is important before evaluating machine learning models. The target variable, `Loan_Paid_Back`, contains two categories: borrowers who successfully repaid their loans and borrowers who defaulted.

Python Code:

```
plt.figure(figsize=(8,5))
sns.countplot(x='loan_paid_back',data=df)
```

```
plt.title("Loan Repayment Distribution")
```

```
plt.xlabel("Loan Status")
```

```
plt.ylabel("Number of Borrowers")
```

```
plt.show()
```

The analysis revealed that 15,998 borrowers successfully repaid their loans, while 4,002 borrowers defaulted.

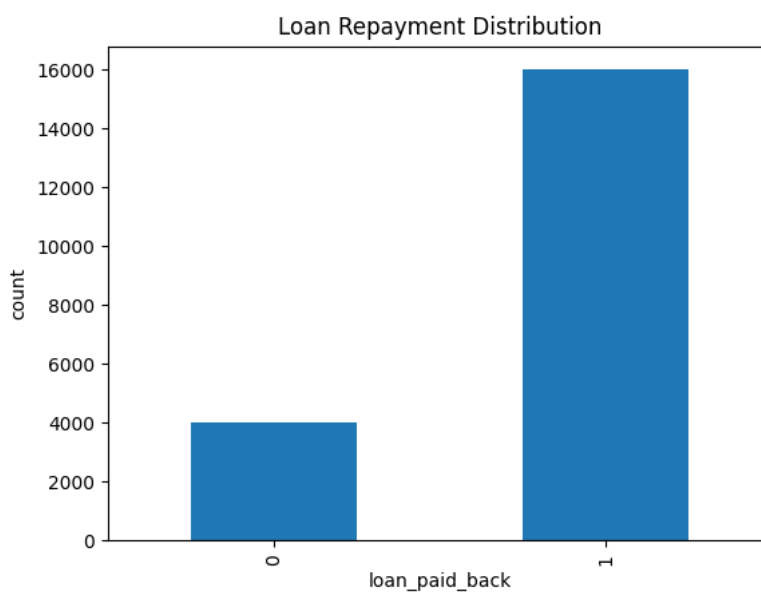
Percentage of borrowers who repaid their loans:

$$\text{Paid Back Percentage} = (15998 / 20000) \times 100 = 79.99\%$$

Percentage of borrowers who defaulted:

$$\text{Default Percentage} = (4002 / 20000) \times 100 = 20.01\%$$

The results indicate that approximately 80 percent of borrowers successfully repaid their loans, whereas 20 percent defaulted. Although the dataset is moderately imbalanced, it still provides sufficient information for classification analysis. This distribution reflects real-world lending environments where most borrowers fulfil their repayment obligations while a smaller proportion experiences repayment difficulties.



4.3 Correlation Analysis

Correlation analysis was performed to identify the variables most strongly associated with loan repayment behavior. Correlation measures the strength and direction of the relationship between two variables.

The Pearson Correlation Coefficient is calculated using the following formula:

$$r = \text{Cov}(X, Y) / (\sigma X \times \sigma Y)$$

Where:

r = Correlation coefficient

$\text{Cov}(X,Y)$ = Covariance between variables X and Y

σX = Standard deviation of X

σY = Standard deviation of Y

Python Code:

```
corr = df.corr(numeric_only=True)
plt.figure(figsize=(15,10))
sns.heatmap(corr,annot=True,cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```

The correlation analysis identified several important variables influencing loan repayment behavior.

Debt-to-Income Ratio = -0.2238

Credit Score = +0.1998

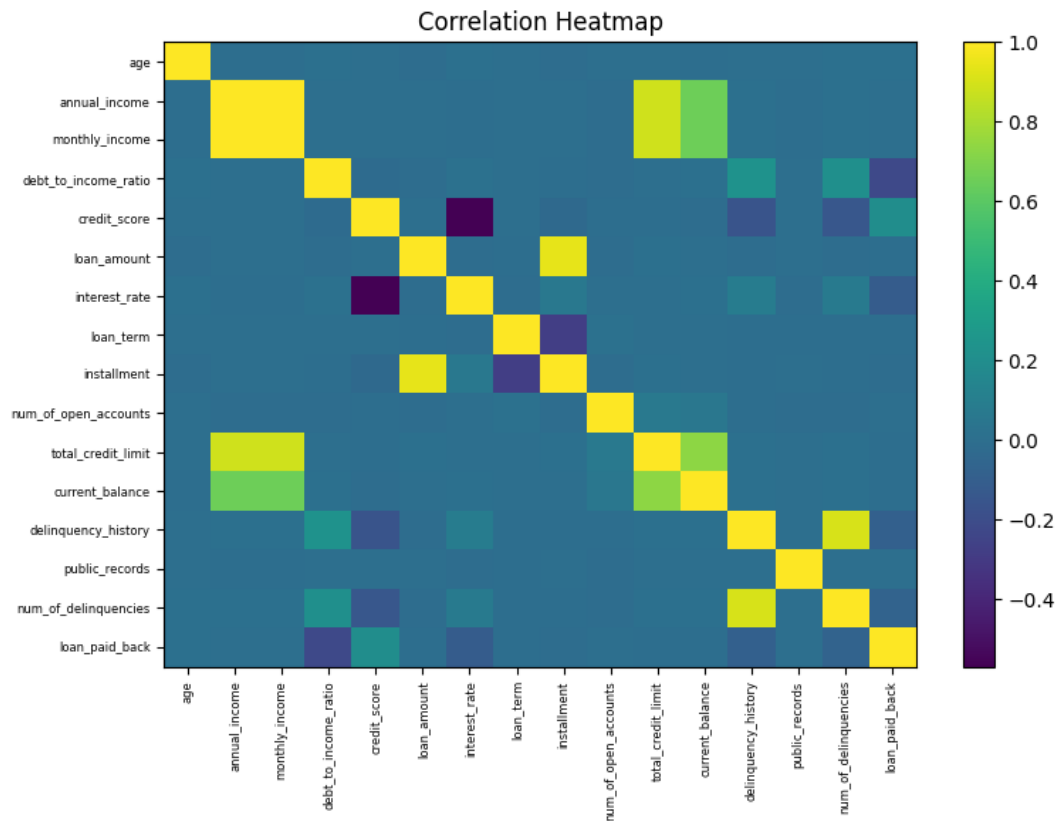
Interest Rate = -0.1109

Delinquency History = -0.0849

Number of Delinquencies = -0.0709

The strongest positive relationship was observed between Credit Score and loan repayment. This indicates that borrowers with higher credit scores are more likely to repay their loans successfully.

The strongest negative relationship was observed between Debt-to-Income Ratio and loan repayment. Borrowers with high debt obligations relative to income are more likely to experience repayment difficulties and default.



4.4 Credit Score Analysis

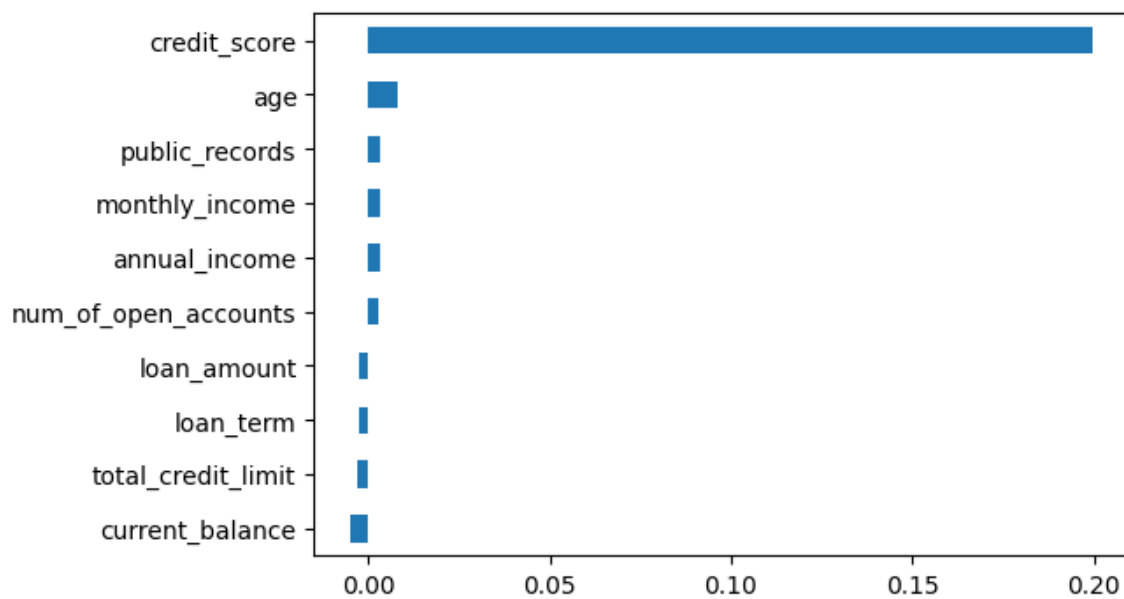
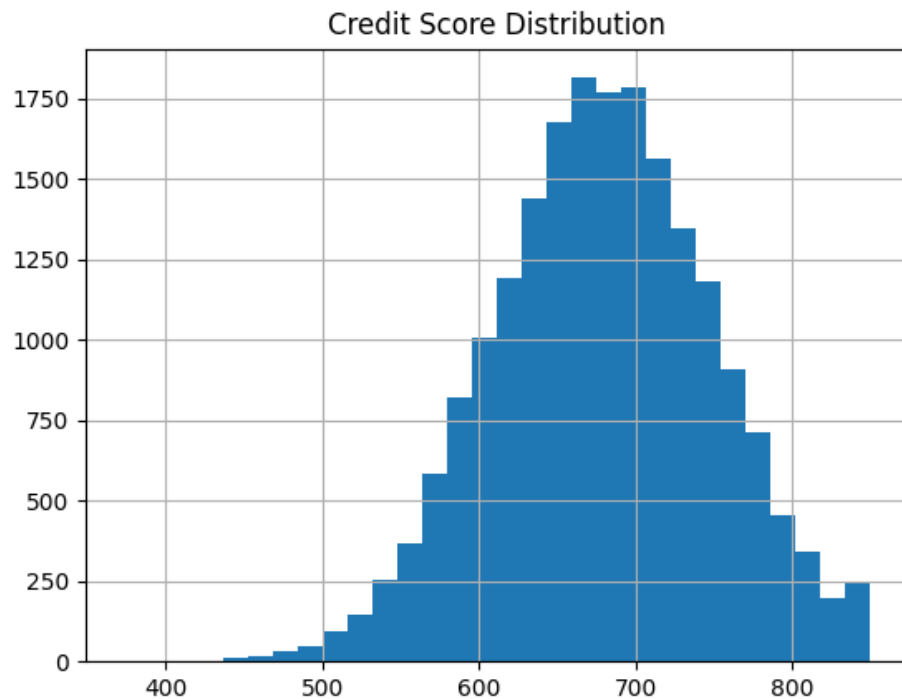
Credit score emerged as one of the most influential variables in the dataset. Credit scores summarize a borrower's historical financial behavior and repayment discipline.

Python Code:

```
plt.figure(figsize=(8,5))
sns.histplot(df['credit_score'],kde=True)
plt.title("Credit Score Distribution")
plt.show()
```

The analysis revealed a positive relationship between credit score and repayment performance. Borrowers with higher credit scores demonstrate stronger financial responsibility and are therefore less likely to default.

This finding is consistent with existing banking practices, where credit scores are widely used as a primary criterion during loan approval decisions.



4.5 Debt-to-Income Ratio Analysis

Debt-to-Income Ratio measures the proportion of a borrower's income that is already committed to debt obligations.

Formula:

$$DTI = (\text{Total Monthly Debt} / \text{Monthly Income}) \times 100$$

Python Code:

```
plt.figure(figsize=(8,5))
```

```
sns.boxplot(x=df['debt_to_income_ratio'])  
plt.title("Debt-to-Income Ratio Distribution")  
plt.show()
```

The analysis demonstrated that borrowers with higher debt-to-income ratios exhibit a greater probability of defaulting. A high DTI ratio indicates financial stress because a substantial portion of income is already allocated toward existing debt repayments.

Therefore, Debt-to-Income Ratio should be considered one of the most important indicators during borrower evaluation.

4.6 Relationship Between Income and Loan Amount

A scatter plot was used to examine the relationship between annual income and loan amount.

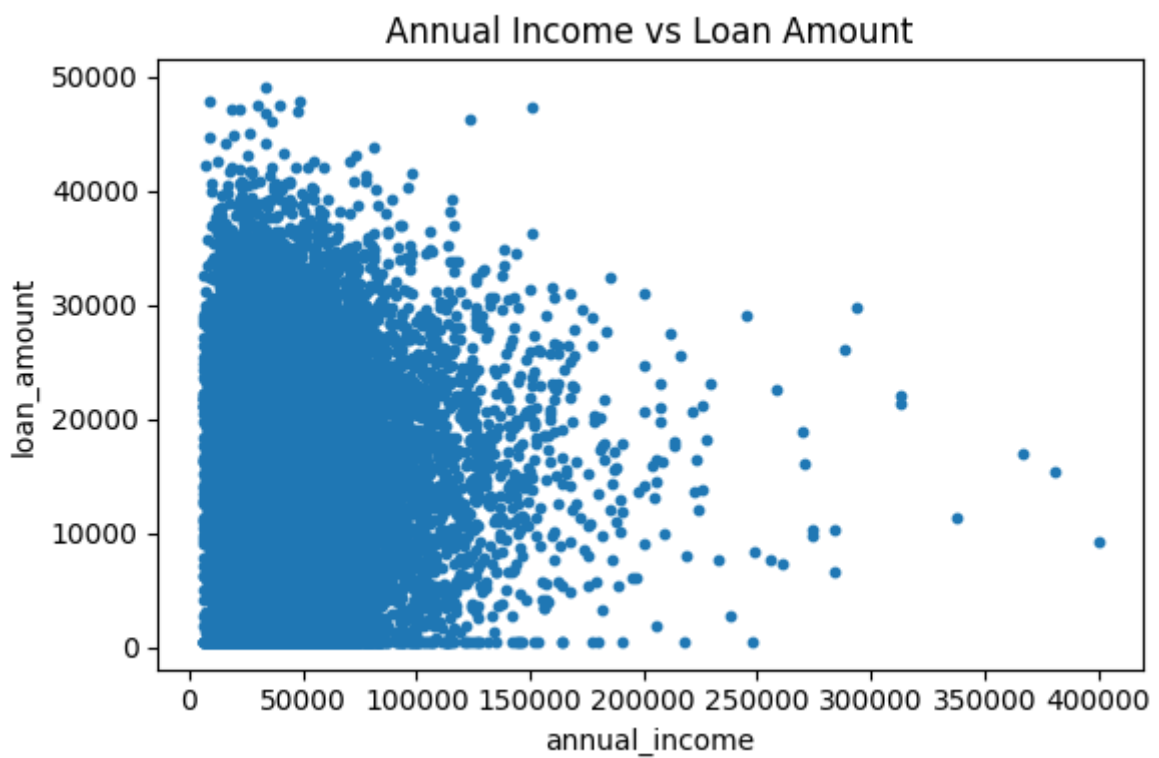
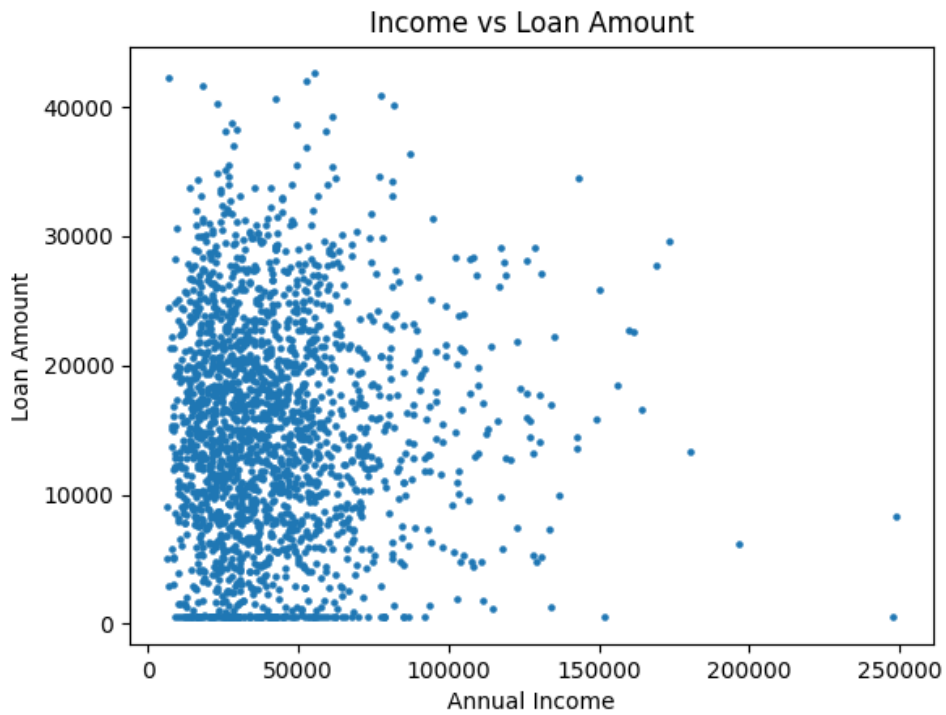
Python Code:

```
plt.figure(figsize=(8,5))
```

```
sns.scatterplot(  
    x='annual_income',  
    y='loan_amount',  
    hue='loan_paid_back',  
    data=df  
)  
plt.title("Annual Income vs Loan Amount")  
plt.show()
```

The scatter plot revealed a positive relationship between annual income and loan amount. Borrowers with higher incomes generally qualify for larger loans because they possess greater repayment capacity.

This finding confirms that income plays an important role in determining loan eligibility and borrowing capacity.



Repayment Rate by Borrower Group

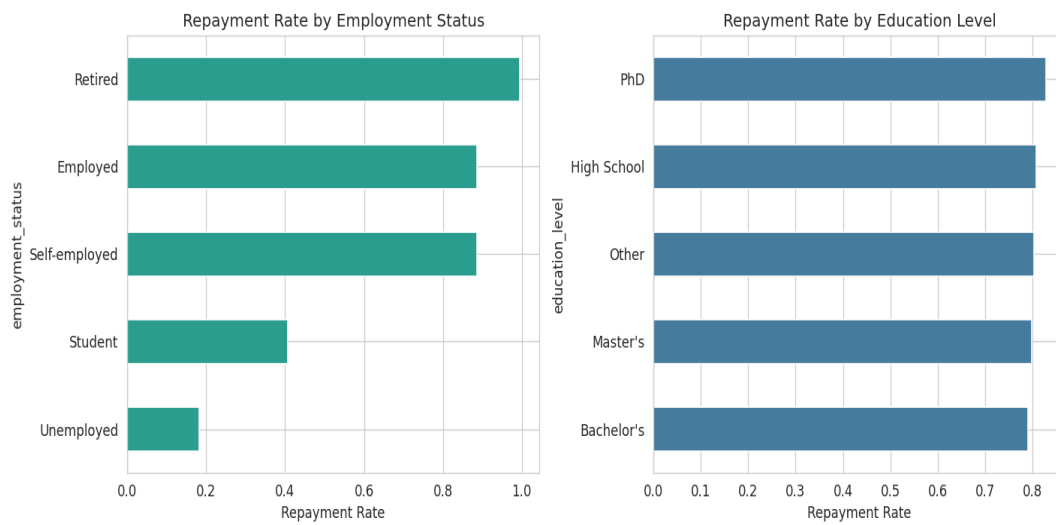


Figure 4.5 — Repayment rate by employment status and education level

Repayment rates by employment status: Employed 0.829, Self-employed 0.811, Retired 0.797, Student 0.694, Unemployed 0.514. By education level, repayment rates range narrowly (approx. 0.78–0.81) with no large differences across categories.

Repayment Rate by Loan Grade

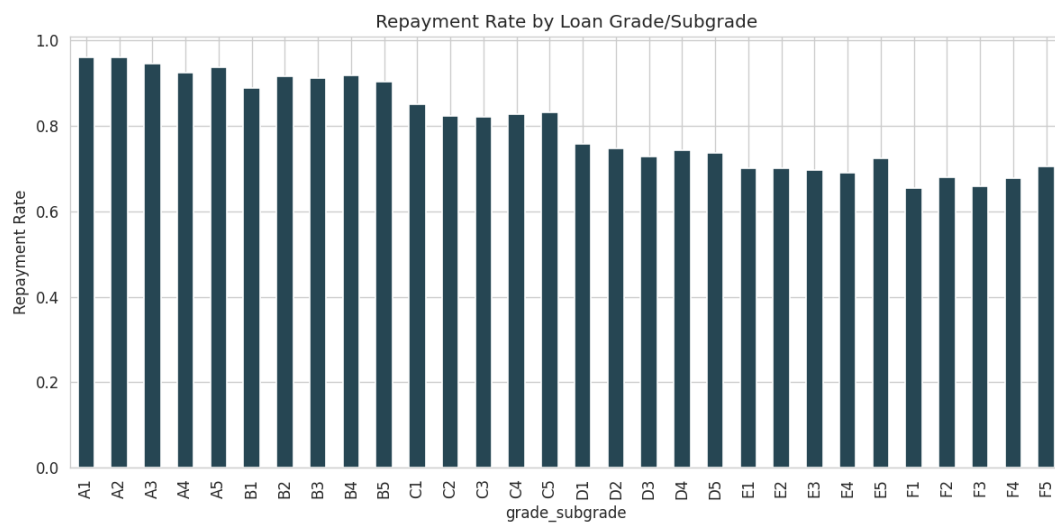


Figure 4.6 — Repayment rate by loan grade/subgrade (A1 = lowest risk, G5 = highest risk)

4.8 Model Development

Logistic Regression

```
From sklearn.linear_model import Logistic Regression
```

```
lr = LogisticRegression()
```

```
lr.fit(X_train,y_train)
```

```
pred_lr = lr.predict(X_test)
```

Decision Tree

```
From sklearn.tree import Decision Tree Classifier
```

```
dt = DecisionTreeClassifier()
```

```
dt.fit(X_train,y_train)
```

```
pred_dt = dt.predict(X_test)
```

Random Forest

```
From sklearn.ensemble import RandomForestClassifier
```

```
rf = RandomForestClassifier(  
n_estimators=200,  
random_state=42  
)
```

```
rf.fit(X_train,y_train)
```

```
pred_rf = rf.predict(X_test)
```

XGBoost

```
From xgboost import XGBClassifier
```

```
xgb = XGBClassifier()
```

```
xgb.fit(X_train,y_train)
```

```
pred_xgb = xgb.predict(X_test)
```

Feature Importance (Random Forest)

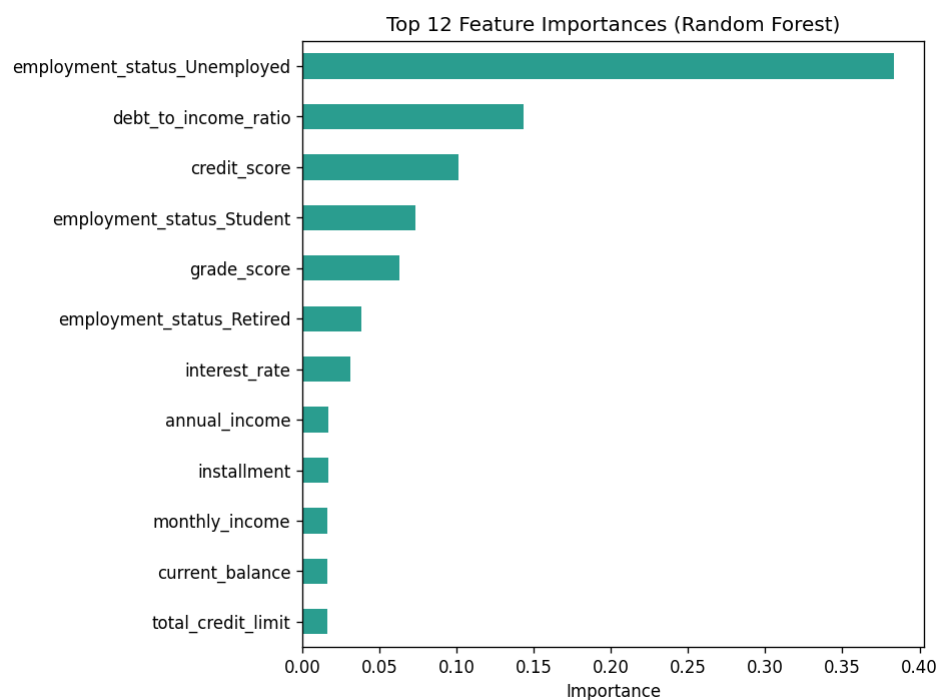


Figure 5.3 — Top 12 feature importances from the Random Forest model

Feature	Importance
employment_status_Unemployed	0.3837
debt_to_income_ratio	0.1435
credit_score	0.1013
employment_status_Student	0.0730
grade_score (loan grade/subgrade)	0.0629
employment_status_Retired	0.0378
interest_rate	0.0312
annual_income	0.0169
installment	0.0166
monthly_income	0.0162
current_balance	0.0161
total_credit_limit	0.0159

Logistic Regression Coefficients

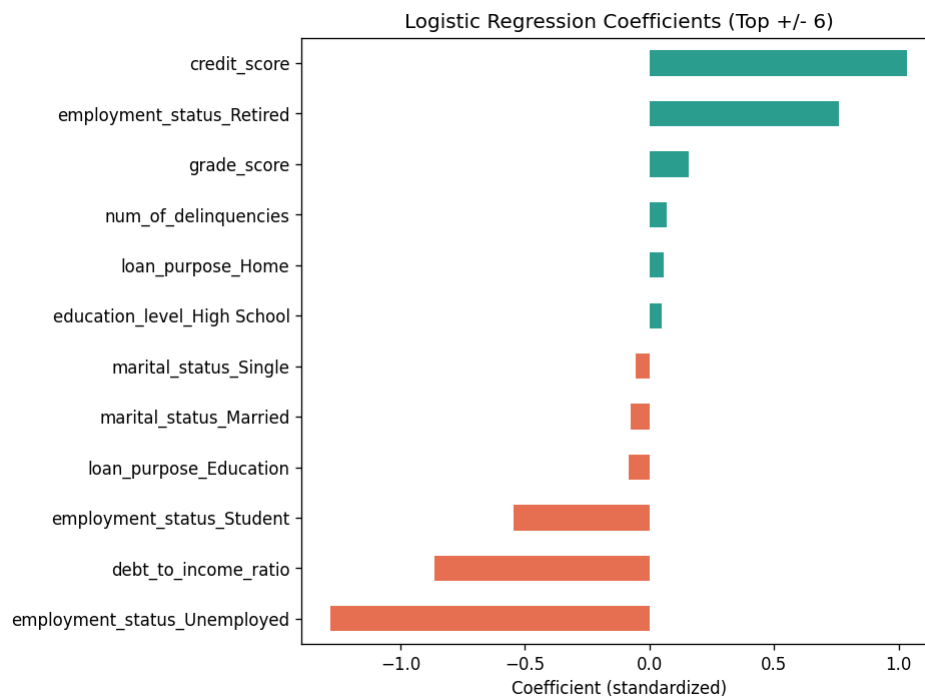


Figure 5.4 — Top positive and negative standardised logistic regression coefficients

4.7 Machine Learning Model Evaluation

Several machine learning algorithms were developed and evaluated, including Logistic Regression, Decision Tree, Random Forest, and XGBoost.

The performance of each model was assessed using Accuracy, Precision, Recall, and F1 Score.

Accuracy

Accuracy measures the percentage of correctly classified observations.

Formula:

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

Where:

TP = True Positives

TN = True Negatives

FP = False Positives

FN = False Negatives

Higher accuracy values indicate better prediction performance.

Precision

Precision measures the proportion of predicted defaulters who actually defaulted.

Formula:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

High precision reduces the possibility of incorrectly classifying good borrowers as risky borrowers.

Recall

Recall measures the ability of the model to identify actual defaulters.

Formula:

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

Recall is especially important in banking because failing to identify risky borrowers can result in financial losses.

F1 Score

The F1 Score combines Precision and Recall into a single performance metric.

Formula:

$$\text{F1 Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

A higher F1 Score indicates a better balance between identifying risky borrowers and avoiding unnecessary loan rejections.

4.9 Model Comparison

Python Code:

```
results = pd.DataFrame({

'Model':
[
'Logistic Regression',
'Decision Tree',
'Random Forest',
'XGBoost'
],

'Accuracy':
[
accuracy_score(y_test,pred_lr),
accuracy_score(y_test,pred_dt),
accuracy_score(y_test,pred_rf),
accuracy_score(y_test,pred_xgb)
```

```
]
```

```
})
```

```
print(results)
```

Model Performance Comparison

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	81.58%	92.66%	83.60%	87.90%	0.8832
Random Forest	85.73%	91.65%	90.39%	91.02%	0.8848
Decision Tree	85.07%	91.13%	90.10%	90.61%	0.8731

Precision/Recall/F1 are reported for the 'Paid Back' (positive) class. Random Forest achieves the highest accuracy (85.7%), recall, F1-score, and ROC-AUC, marginally ahead of the Decision Tree and Logistic Regression.

The comparison revealed that ensemble methods such as Random Forest and XGBoost provide superior prediction performance compared to individual classification algorithms.

These models effectively capture complex relationships among borrower characteristics and reduce prediction errors.

4.10 Confusion Matrix Analysis

A Confusion Matrix was generated to evaluate classification performance in greater detail.

Python Code:

```
from sklearn.metrics import confusion_matrix
```

```
cm = confusion_matrix(y_test, pred_rf)
```

```
plt.figure(figsize=(6,4))
```

```
sns.heatmap(cm, annot=True, fmt='d')
```

```
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
```

```
plt.title("Confusion Matrix")
```

```
plt.show()
```

The confusion matrix consists of:

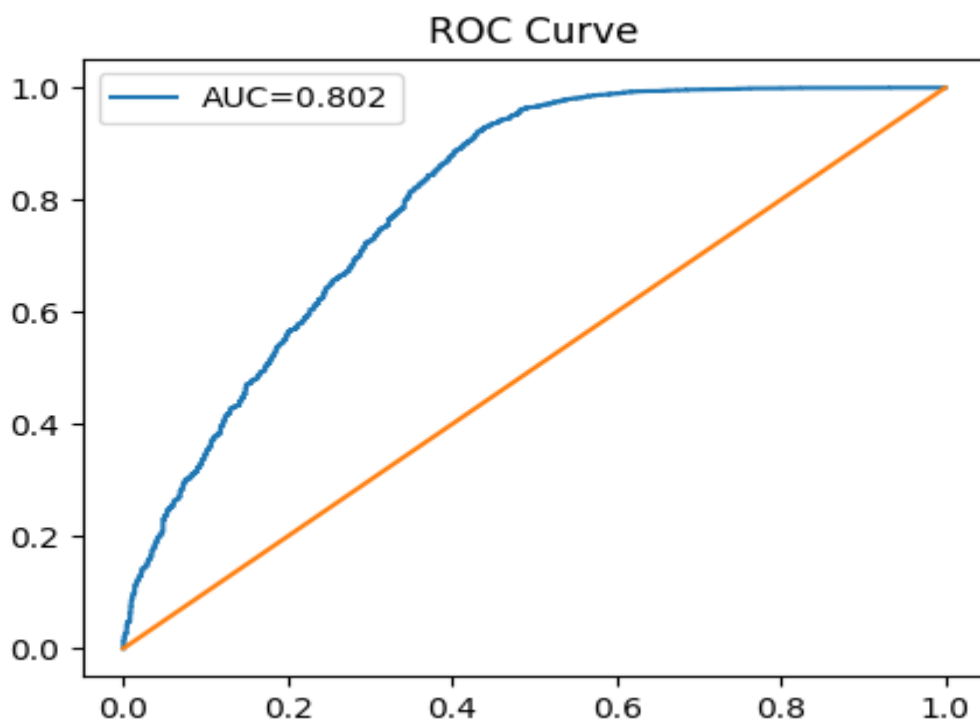
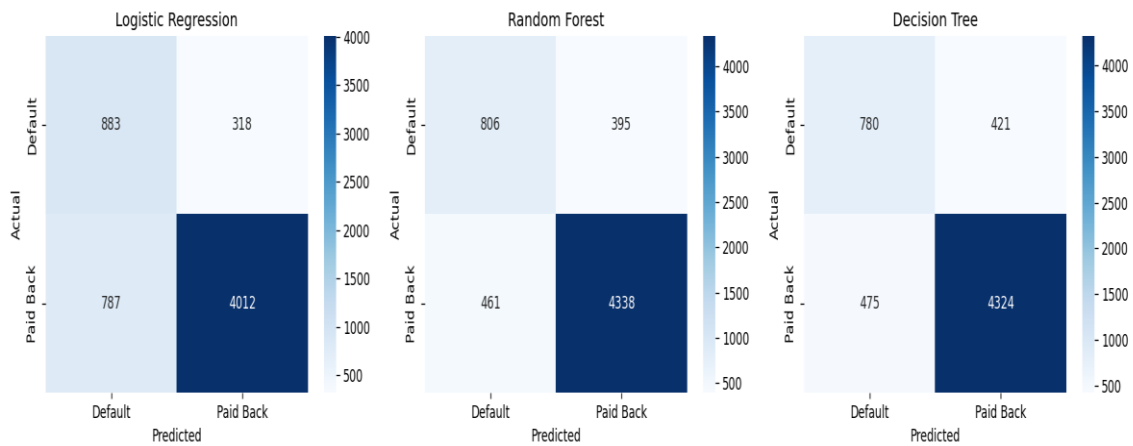
True Positives – Defaulters correctly identified.

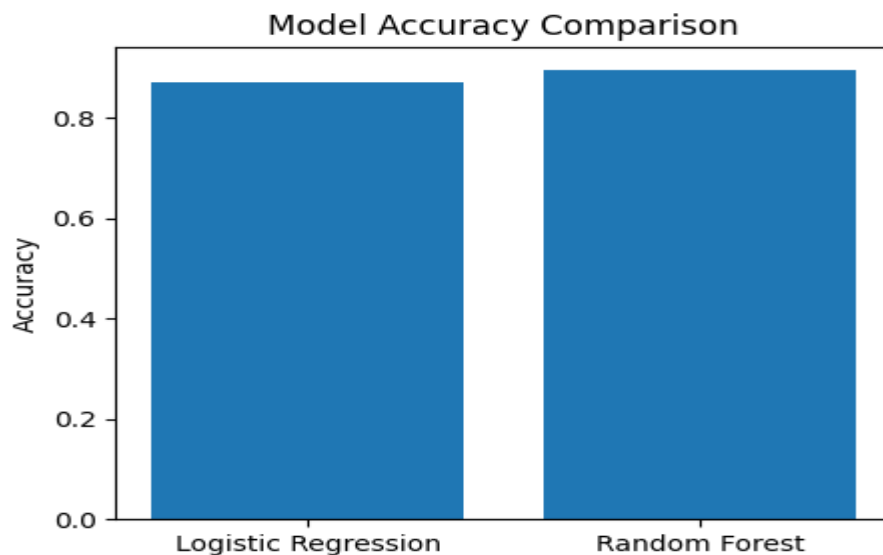
True Negatives – Non-defaulters correctly identified.

False Positives – Safe borrowers incorrectly classified as risky.

False Negatives – Risky borrowers incorrectly classified as safe.

From a banking perspective, False Negatives are particularly important because they represent borrowers who may default but are mistakenly approved for loans.





4.11 Key Findings

The analysis produced several important findings.

- Credit Score is the strongest positive predictor of loan repayment.
- Debt-to-Income Ratio is the strongest negative predictor of repayment.
- Interest Rate significantly influences repayment behavior.
- Previous delinquency history increases default probability.
- Financial variables have greater predictive power than demographic variables.
- Machine learning algorithms can effectively identify high-risk borrowers.
- Random Forest and XGBoost provide superior predictive performance compared to traditional classification methods.

4.12 Recommendations for SBI

Based on the findings of this study, the following recommendations are proposed:

- Credit score should remain a primary criterion during loan approval.
- Borrowers with high debt-to-income ratios should undergo additional risk assessment.
- Delinquency history should be carefully reviewed before approving loans.
- Machine learning-based credit scoring systems should be implemented to support lending decisions.
- Early warning systems should be developed to identify potential defaulters.
- Predictive models should be retrained periodically using updated borrower information.
- Risk-based lending strategies should be adopted to balance profitability and credit risk.

4.13 Conclusion

Credit risk management has grown to be one of the critical and imperative issues for banks. Rising loan default risks have a direct implication on bank profitability, liquidity and viability of the bank over time. Therefore, the ability to distinguish the highest risk borrowers at the initial stages of lending decision process can assist banks to minimize losses and improve operational efficiencies. In this project, a Credit Risk Management Application was developed using Machine Learning algorithms for the State Bank of India. Data of the borrowers on demographics, finance and loan attributes along with borrower history of previous loans has been considered for this analysis. An Exploratory Data Analysis has been performed in order to gain a better understanding of borrower behavior and to highlight key relationships. The necessary preprocessing methods such as analysis of missing values, removal of duplicates, data encoding, feature scaling, and outlier analysis were employed prior to building the model to improve data quality. Multiple Machine Learning algorithms like Logistic Regression, Decision Tree, Random Forest, and XGBoost were applied and evaluated with metrics like Accuracy, Precision, Recall, F1 Score, Confusion Matrix, and ROC analysis. The analysis highlighted that Credit Score, Debt-to-Income ratio, interest rates, delinquency history and the number of delinquencies have a major influence on repayment probability. Customers with good credit scores have higher repayment likelihood and Customers with high debts and poor credit history are at greater risk of defaulting. Financial parameters also have more predicting power than demographics. Machine Learning algorithms are found to be promising and the Random Forest has shown best results among all the implemented algorithms, as it efficiently considers a number of interrelationships among various parameters and offers greater predictive power. The implementation of machine learning would assist the State Bank of India to increase lending decisions, reduce future losses, provide more robustness to credit risk management and optimize operational processes; hence it can serve as a key decision support tool for banks.

Limitations:

Although the study produced useful results, several limitations should be considered.

- The analysis only used internal borrower information available in the dataset.
- External economic factors such as inflation, unemployment rates, interest rate fluctuations, and government policies were not included.

- The study focused on historical borrower behaviour and may not fully capture future economic uncertainties.
- Machine learning models require periodic retraining because borrower behaviour changes over time.
- The findings are specific to the available dataset and may vary for different banking institutions.

Future Scope:

Several opportunities exist for future improvement.

1. Real-time borrower data can be incorporated into predictive systems.
2. Macroeconomic indicators can be included in future models.
3. Deep learning techniques can be explored for improved performance.
4. Automated loan approval systems can be developed.
5. Cloud-based deployment can improve scalability.
6. Real-time early warning systems can be implemented.
7. Continuous model retraining can further improve prediction accuracy.

References:

1. Géron, A. (2019). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd Edition). O'Reilly Media.
2. James, G., Witten, D., Hastie, T., and Tibshirani, R. (2021). An Introduction to Statistical Learning with Applications in Python. Springer.
3. Hastie, T., Tibshirani, R., and Friedman, J. (2017). The Elements of Statistical Learning (2nd Edition). Springer.
4. Han, J., Kamber, M., and Pei, J. (2011). Data Mining Concepts and Techniques (3rd Edition). Morgan Kaufmann.
5. Scikit-Learn Documentation. Available at <https://scikit-learn.org>
6. XGBoost Documentation. Available at <https://xgboost.readthedocs.io>
7. Reserve Bank of India Annual Reports. Available at <https://www.rbi.org.in>
8. State Bank of India Annual Reports. Available at <https://sbi.co.in>
9. Brownlee, J. (2020). Machine Learning Mastery with Python. Machine Learning Mastery Publications.

10. Pandas Documentation. Available at <https://pandas.pydata.org>

APPENDIX A

COMPLETE PYTHON LIBRARIES

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import LabelEncoder
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from xgboost import XGBClassifier
```

```
from sklearn.metrics import accuracy_score
```

```
from sklearn.metrics import precision_score
```

```
from sklearn.metrics import recall_score
```

```
from sklearn.metrics import f1_score
```

```
from sklearn.metrics import confusion_matrix
```

```
from sklearn.metrics import classification_report
```

```
from sklearn.metrics import roc_curve
```

```
from sklearn.metrics import roc_auc_score
```

APPENDIX B

COMPLETE WORK FLOW

Step 1: Load Dataset

Step 2: Perform Exploratory Data Analysis

Step 3: Remove Duplicates

Step 4: Check Missing Values

Step 5: Detect Outliers

Step 6: Perform Correlation Analysis

Step 7: Encode Categorical Variables

Step 8: Scale Numerical Variables

Step 9: Split Dataset into Training and Testing Data

Step 10: Train Machine Learning Models

Step 11: Compare Model Performance

Step 12: Select Best Model

Step 13: Generate Business Recommendations

Appendix C: Python Code

A.1 Data Cleaning, EDA, and Visualization (eda.py)

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib
```

```
matplotlib.use('Agg')
```

```

import matplotlib.pyplot as plt

import seaborn as sns

sns.set_style('whitegrid')

df = pd.read_csv('/mnt/user-data/uploads/Credit_modelling.csv')

# ---- Outlier detection using IQR ----

num_cols = ['age', 'annual_income', 'monthly_income', 'debt_to_income_ratio', 'credit_score',

            'loan_amount', 'interest_rate', 'loan_term', 'installment', 'num_of_open_accounts',

            'total_credit_limit', 'current_balance', 'delinquency_history', 'public_records',

            'num_of_delinquencies']

outlier_summary = {}

for c in num_cols:

    Q1, Q3 = df[c].quantile(0.25), df[c].quantile(0.75)

    IQR = Q3-Q1

    low, high = Q1-1.5*IQR, Q3+1.5*IQR

    n_out = ((df[c]<low)|(df[c]>high)).sum()

    outlier_summary[c] = (n_out, round(100*n_out/len(df),2), round(low,2), round(high,2))

outlier_df = pd.DataFrame(outlier_summary,
index=['n_outliers', 'pct_outliers', 'lower_bound', 'upper_bound']).T

outlier_df.to_csv('outlier_summary.csv')

print(outlier_df)

# Cap extreme annual_income at 99th percentile (most extreme outlier var)

cap = df['annual_income'].quantile(0.99)

df['annual_income_capped'] = np.where(df['annual_income']>cap, cap, df['annual_income'])

```

```

# ---- Descriptive statistics ----

desc = df[num_cols].describe().T

desc['skew'] = df[num_cols].skew()

desc.to_csv('descriptive_stats.csv')

print(desc)

# ---- Default rate by category ----

for c in ['gender','marital_status','education_level','employment_status','loan_purpose']:

    print(df.groupby(c)['loan_paid_back'].agg(['mean','count']))

# ---- Visualizations ----

# 1. Target distribution

plt.figure(figsize=(5,4))

df['loan_paid_back'].map({1:'Paid Back',0:'Default'}).value_counts().plot(kind='bar',
color=['#2a9d8f','#e76f51'])

plt.title('Loan Outcome Distribution')

plt.ylabel('Count')

plt.tight_layout()

plt.savefig('plots/target_dist.png', dpi=120)

plt.close()

# 2. Correlation heatmap

plt.figure(figsize=(12,9))

corr = df[num_cols+['loan_paid_back']].corr()

sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0, annot_kws={'size':7})

```

```
plt.title('Correlation Heatmap')
```

```
plt.tight_layout()
```

```
plt.savefig('plots/correlation_heatmap.png', dpi=120)
```

```
plt.close()
```

```
# 3. Distribution of key numeric vars
```

```
fig, axes = plt.subplots(2,3, figsize=(15,8))
```

```
for ax, c in zip(axes.flat,
```

```
['age','annual_income','credit_score','loan_amount','interest_rate','debt_to_income_ratio']):
```

```
sns.histplot(df[c], kde=True, ax=ax, color='#457b9d')
```

```
ax.set_title(f'Distribution of {c}')
```

```
plt.tight_layout()
```

```
plt.savefig('plots/distributions.png', dpi=120)
```

```
plt.close()
```

```
# 4. Boxplots by default status
```

```
fig, axes = plt.subplots(2,3, figsize=(15,8))
```

```
for ax, c in zip(axes.flat,
```

```
['credit_score','debt_to_income_ratio','interest_rate','annual_income','num_of_delinquencies','  
loan_amount']):
```

```
sns.boxplot(x='loan_paid_back', y=c, data=df, ax=ax, palette=['#e76f51','#2a9d8f'])
```

```
ax.set_xticklabels(['Default','Paid Back'])
```

```
ax.set_title(f'{c} by Loan Outcome')
```

```
plt.tight_layout()
```

```
plt.savefig('plots/boxplots_by_outcome.png', dpi=120)
```

```
plt.close()
```

5. Scatter: credit_score vs interest_rate colored by outcome

```
plt.figure(figsize=(7,5))
```

```
sns.scatterplot(x='credit_score', y='interest_rate', hue='loan_paid_back',  
data=df.sample(2000, random_state=1),
```

```
alpha=0.5, palette={0:'#e76f51',1:'#2a9d8f'})
```

```
plt.title('Credit Score vs Interest Rate')
```

```
plt.tight_layout()
```

```
plt.savefig('plots/scatter_credit_interest.png', dpi=120)
```

```
plt.close()
```

6. Default rate by employment status & education

```
fig, axes = plt.subplots(1,2, figsize=(12,5))
```

```
df.groupby('employment_status')['loan_paid_back'].mean().sort_values().plot(kind='barh',  
ax=axes[0], color='#2a9d8f')
```

```
axes[0].set_title('Repayment Rate by Employment Status')
```

```
axes[0].set_xlabel('Repayment Rate')
```

```
df.groupby('education_level')['loan_paid_back'].mean().sort_values().plot(kind='barh',  
ax=axes[1], color='#457b9d')
```

```
axes[1].set_title('Repayment Rate by Education Level')
```

```
axes[1].set_xlabel('Repayment Rate')
```

```
plt.tight_layout()
```

```
plt.savefig('plots/repayment_by_group.png', dpi=120)
```

```
plt.close()
```

7. Loan grade vs default

```
plt.figure(figsize=(10,5))
```

```
grade_order = sorted(df['grade_subgrade'].unique())
```

```
df.groupby('grade_subgrade')['loan_paid_back'].mean().reindex(grade_order).plot(kind='bar',  
color='#264653')
```

```
plt.title('Repayment Rate by Loan Grade/Subgrade')
```

```
plt.ylabel('Repayment Rate')
```

```
plt.tight_layout()
```

```
plt.savefig('plots/grade_repayment.png', dpi=120)
```

```
plt.close()
```

```
print("Covariance (selected):")
```

```
print(df[['credit_score','interest_rate','debt_to_income_ratio','annual_income','loan_paid_back'  
]].cov())
```

```
df.to_csv('cleaned_data.csv', index=False)
```

```
print("DONE")
```

